Open in Colab

Intermediate Machine Learning: Assignment 5

Deadline

Assignment 5 is due Thursday, December 4 by 11:59pm. Late work will not be accepted as per the course policies (see the Syllabus and Course policies on Canvas).

Directly sharing answers is not okay, but discussing problems with the course staff or with other students is encouraged. Acknowledge any use of an AI system such as ChatGPT or Copilot.

You should start early so that you have time to get help if you're stuck. The drop-in office hours schedule can be found on Canvas. You can also post questions or start discussions on Ed Discussion. The assignment may look long at first glance, but the problems are broken up into steps that should help you to make steady progress.

Submission

Submit your assignment as a pdf file on Gradescope, and as a notebook (.ipynb) on Canvas. You can access Gradescope through Canvas on the left-side of the class home page. The problems in each homework assignment are numbered. Note: When submitting on Gradescope, please select the correct pages of your pdf that correspond to each problem. This will allow graders to more easily find your complete solution to each problem.

To produce the .pdf, please convert to html and then print to pdf. (You may want to use your pdf print menu to scale the pages to be sure that cells are not truncated.) To convert to html, you can use this converter notebook.

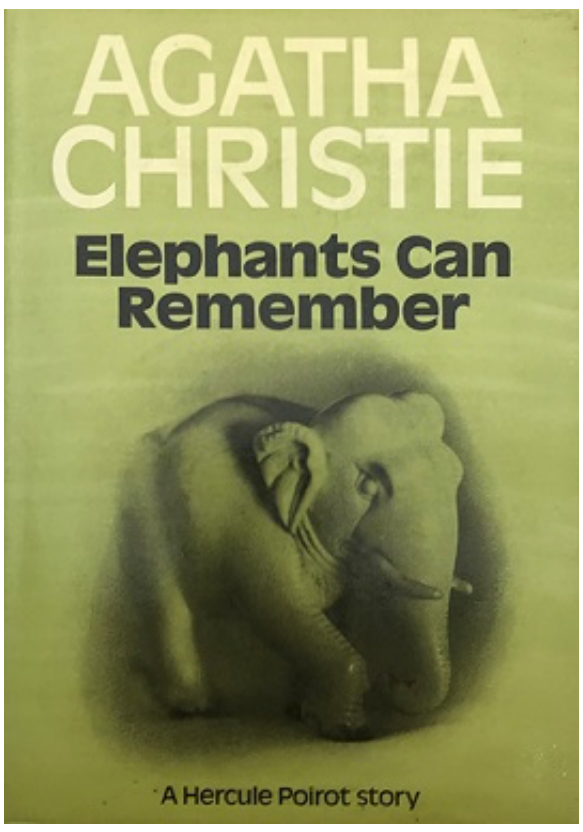
You should start early so that you have time to get help if you're stuck. The drop-in office hours schedule can be found on Canvas. You can also post questions or start discussions on Ed Discussion. The assignment may look long at first glance, but the problems are broken up into steps that should help you to make steady progress.

Topics

- RNNs and GRUs
- Transformers

After doing this assignment you will have a working knowledge of RNNs and Transformers, and more solid Python skills.

Problem 1: Elephants Can Remember (25 points)



In this problem, we will work with "vanilla" Recurrent Neural Networks (RNNs) and Recurrent Neural Networks with Gated Recurrent Units (GRUs). The models in this part of the assignment will be character-based models, trained on an extract of the book Elephants Can Remember by Agatha Christie. To reduce the size of our vocabulary, the text is pre-processed by converting the letters to lower case and removing numbers. The code below shows some information about our training and test set. All the necessary files for this problem are available on github.

In [1]:
```
import numpy as np
```

```
from tqdm import tqdm
import tensorflow as tf
import random
import json

# Ensure tf.keras is the main keras module
# import tensorflow.keras as keras # This line is no longer strictly necessary if all components are explic

from tensorflow.keras.models import Sequential, model_from_json
from tensorflow.keras.layers import Dense, Activation
from tensorflow.keras.layers import GRU, SimpleRNN
```

In [2]:
```
from google.colab import drive
drive.mount('/content/drive')
data_path = '/content/drive/MyDrive/assn5/'
               # set to the path where you store the assignment files from github
               # e.g., '/content/drive/MyDrive/assn5/'
```

```
Mounted at /content/drive
```

In [3]:
```
with open(data_path + 'gru_problem/Agatha_Christie_train.txt', 'r') as file:
    train_text = file.read()

with open(data_path + 'gru_problem/Agatha_Christie_test.txt', 'r') as file:
    test_text = file.read()

vocabulary = sorted(list(set(train_text + test_text)))
vocab_size = len(vocabulary)

# Dictionaries to go from a character to index and vice versa
char_to_indices = dict((c, i) for i, c in enumerate(vocabulary))
indices_to_char = dict((i, c) for i, c in enumerate(vocabulary))
```

In [4]:
```
# The first 500 characters of our training set
train_text[0:500]
```

Out[4]:
```
'mrs. oliver looked at herself in the glass. she gave a brief, sideways look towards the clock on the mant
elpiece, which she had some idea was twenty minutes slow. then she resumed her study of her coiffure. the
trouble with mrs. oliver was--and she admitted it freely--that her styles of hairdressing were always bein
g changed. she had tried almost everything in turn. a severe pompadour at one time, then a wind-swept styl
e where you brushed back your locks to display an intellectual brow, at least'
```

In [5]:
```
print("The vocabulary contains", vocab_size, "characters")
print("The training set contains", len(train_text) ,"characters")
print("The test set contains", len(test_text) ,"characters")
```

```
The vocabulary contains 44 characters
The training set contains 262174 characters
The test set contains 7209 characters
```

Problem 1.1: The Diversity of Language Models

Before jumping into coding, let's start with comparing the language models we will be using in this assigment.

1. Describe the differences between a Vanilla RNN and a GRU network. In your explanation, make sure you mention the issues with vanilla RNNs and how GRUs try to solve them.

- In Vanilla RNNs, at each time step t, the hidden state h_t is computed from the current input x_t and the previous hidden state h_{t-1} and each step combines the state of the current input and all the historical information.
- But during backpropagation through long sequences, gradients can vanish or explode, which makes it hard for Vanilla RNNs to actually remember all the historical information.
- The GRU uses gates to solve these issues.
  - The update gate: When open, information from long-term memory is propagated. When closed, information from local state is used. It decides how much past information should be kept and how much current state should be accepted.
  - The reset gate: When closed, the local state is reset. When open, the state is updated as in a Vanilla RNN. It decides how much past information should be forgotten.
  - It can help reduce the problem of gradient vanishing.

2. Describe at least two advantages of a character based language model over a word based language model.

- A character based language model can better deal with words that do not appear in the vacabulary, since it simply divides them to be single characters, while a word based language model will be unknown of those words.

- A character based language model needs smaller vocabulary size than a word based language model, which can reduce the cost of computation and avoids sparsity.

Problem 1.2: Generating Text with the Vanilla RNN

The code below loads in a pretrained vanilla RNN model with two layers. The model is set up exactly like in the lecture slides (with tanh activation layers in the recurrent layers) with the addition of biases (intercepts) in every layer (i.e. the recurrent layer and the dense layer). The training process consisted of 30 epochs.

In [6]:
```
# load json and create model
json_file = open(data_path + 'gru_problem/RNN_model.json', 'r')
loaded_model_json_str = json_file.read()
json_file.close()

RNN_model = model_from_json(loaded_model_json_str, custom_objects={'Sequential': Sequential, 'SimpleRNN': S
RNN_model.load_weights(data_path + "gru_problem/RNN_model.h5")
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input
_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` objec
t as the first layer in the model instead.
  super().__init__(**kwargs)
```

In [7]:
```
# load in the weights and show summary
weights_RNN = RNN_model.get_weights()
RNN_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
Vanilla_RNN_1 (SimpleRNN)	(None, 100, 128)	22,144
Vanilla_RNN_2 (SimpleRNN)	(None, 64)	12,352
Dense_layer (Dense)	(None, 44)	2,860
Softmax_layer (Activation)	(None, 44)	0

Total params: 37,356 (145.92 KB)
Trainable params: 37,356 (145.92 KB)
Non-trainable params: 0 (0.00 B)

Finish the following function that uses a vanilla RNN architecture to generate text, given the weights of the RNN model, a text prompt, and the number of characters to return. The function should be completed by **only using numpy functions**. Use your knowledge of how every weight plays its role in the RNN architecture. Do not worry about the weight extraction part, this is already provided for you. The weight matrix W_{xh1}, for example, denotes the weight matrix to go from the input x to the first hidden state layer h1. The hidden states h_1 and h_2 are initialized to a vector of zeros.

The embedding of each character has to be done by a one-hot encoding, where you will need the dictionaries defined in the introduction to go from a character to an index position.

In [8]:
```
def sample_text_RNN(weights, prompt, N):
    '''
    Uses a pretrained RNN to generate text, starting from a prompt,
    only using the weights and numpy commands
            Parameters:
                    weights (list): Weights of the pretrained RNN model
                    prompt (string): Start of generated sentence
                    N (int): Length of output sentence (including prompt)
            Returns:
                    output_sentence (string): Text generated by RNN
    '''
    # Extracting weights and biases
    # Dimensions of matrices are same format as lecture slides

    # First Recurrent Layer
    W_xh1 = weights[0].T
    W_h1h1 = weights[1].T
    b_h1 = np.expand_dims(weights[2], axis=1)

    # Second Recurrent Layer
    W_h1h2 = weights[3].T
    W_h2h2 = weights[4].T
    b_h2 = np.expand_dims(weights[5], axis=1)

    # Linear (dense) layer
```

```
    W_h2y = weights[6].T
    b_y = np.expand_dims(weights[7], axis=1)

    # Initiate the hidden states
    h1 = np.zeros((W_h1h1.shape[0], 1))
    h2 = np.zeros((W_h2h2.shape[0], 1))

    # ------------------------------------------------

    # Your code starts here
    for ch in prompt:
      x = np.zeros((vocab_size, 1))
      x[char_to_indices[ch]] = 1
      h1 = np.tanh(np.dot(W_h1h1, h1) + np.dot(W_xh1, x) + b_h1)
      h2 = np.tanh(np.dot(W_h2h2, h2) + np.dot(W_h1h2, h1) + b_h2)

    output_sentence = prompt
    current_ch = prompt[-1]

    for i in range(len(prompt), N):
      x = np.zeros((vocab_size, 1))
      x[char_to_indices[current_ch]] = 1
      h1 = np.tanh(np.dot(W_h1h1, h1) + np.dot(W_xh1, x) + b_h1)
      h2 = np.tanh(np.dot(W_h2h2, h2) + np.dot(W_h1h2, h1) + b_h2)
      y = np.dot(W_h2y, h2) + b_y
      p = np.exp(y) / np.sum(np.exp(y))
      current_ch_idx = np.random.choice(vocab_size, p=p.reshape(-1))
      current_ch = indices_to_char[current_ch_idx]
      # current_ch = indices_to_char[np.argmax(p)]
      output_sentence += current_ch

    return output_sentence
```

Test out your function by running the following code cell. Use it as a sanity check that your code is working. The generated text should not be perfect English, but at least you should be able to recognize some words.

In [9]:
```
print(sample_text_RNN(weights_RNN,
                      'mrs. oliver looked at herself in the glass. she gave a brief, sideways look',
                      1000))
```

```
mrs. oliver looked at herself in the glass. she gave a brief, sideways lookn him in the come tadn i mentban
guthay. ine of mrs. bly well hearld is i befert of writing you know who had that there were she's call chil
dy. "one wege in but i don'th wellnge lift you had it's ayty, she'd had anyth! whlens," she said, you being
to say there ofed," said mrs. oliver, "suspicter. ond ut." "and im a mest-ly do not he was or weftren that
surticalald be the do you things. you verded i think she'd sadd of the reveribging. homomusher fare eirly i
t lived foh onchought aborsted mracwase her." "wen why was it's intternt aftive lived headin: "exalle you m
eany it she was he'd radon, you will, i think, were." "yes, you do dot exters and a yound her dang her time
litton the hushe beatin some?" "yes, but most a lettlack of whell, if isulted selinur nevelded, able to eli
ce time for lonkilled your his there say," said poirot to and yesmant?" "wele, really. the samifuirs." "nom
ing. somethan there was a sme?" "ye
```

Problem 1.3: Generating Text with the GRU

The code below loads in a pretrained GRU model. The model is set up exactly like in the lecture slides (with sigmoid activation layers for the gates and tanh activation layers in the recurrent layer). The model is trained for only 10 epochs.

In [10]:
```
# load json and create model
json_file = open(data_path + 'gru_problem/GRU_model.json', 'r')
loaded_model_json_str = json_file.read()
json_file.close()

GRU_model = model_from_json(loaded_model_json_str, custom_objects={'Sequential': Sequential, 'GRU': GRU})
GRU_model.load_weights(data_path + "gru_problem/GRU_model.h5")
```

In [11]:
```
# load in the weights and show summary
weights_GRU = GRU_model.get_weights()
GRU_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
gru (GRU)	(None, 512)	857,088
dense (Dense)	(None, 44)	22,572
activation (Activation)	(None, 44)	0

Total params: 879,660 (3.36 MB)

```
Trainable params: 879,660 (3.36 MB)
Non-trainable params: 0 (0.00 B)
```

Finish the following function that uses a GRU architecture to generate text, given the weights of the GRU model, a text prompt, and the number of characters to return. The function should be completed by **only using numpy functions**. Use your knowledge of how every weight plays its role in the GRU architecture. Do not worry about the weight extraction part, this is already provided for you. The hidden state h is initialized to a vector of zeros.

The embedding of each character has to be done by a one-hot encoding, where you will need the dictionaries defined in the introduction to go from a character to an index position.

Note: a slightly different version of the GRU is used, where the candidate state c_t is calculated as:

$$c_t = \tanh\left(W_{hx}x_t \ + \ \Gamma_t^r \odot (W_{hh}h_{t-1}) \ + \ b_h\right)$$

In [12]:
```
# Helper function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sample_text_GRU(weights, prompt, N):
    '''
    Uses a pretrained GRU to generate text, starting from a prompt,
    only using the weights and numpy commands
        Parameters:
                weights (list): Weights of the pretrained GRU model
                prompt (string): Start of generated sentence
                N (int): Total length of output sentence
        Returns:
                output_sentence (string): Text generated by GRU
    '''
    # Extracting weights and biases
    # Dimensions of matrices are same format as lecture slides

    # GRU Layer
    W_ux, W_rx, W_hx = np.split(weights[0].T, 3, axis = 0)
    W_uh, W_rh, W_hh = np.split(weights[1].T, 3, axis = 0)

    bias = np.sum(weights[2], axis=0)
    b_u, b_r, b_h = np.split(np.expand_dims(bias, axis=1), 3)

    # Linear (dense) layer
    W_y = weights[3].T
    b_y = np.expand_dims(weights[4], axis=1)

    # Initiate hidden state
    h = np.zeros((W_hh.shape[0], 1))

    # ------------------------------------------------

    # Your code starts here
    for ch in prompt:
      x = np.zeros((vocab_size, 1))
      x[char_to_indices[ch]] = 1
      u_t = sigmoid(np.dot(W_ux, x) + np.dot(W_uh, h) + b_u)
      r_t = sigmoid(np.dot(W_rx, x) + np.dot(W_rh, h) + b_r)
      c_t = np.tanh(np.dot(W_hx, x) + r_t * np.dot(W_hh, h) + b_h)
      h = (1 - u_t) * c_t + u_t * h

    output_sentence = prompt
    current_ch = prompt[-1]

    for i in range(len(prompt), N):
      x = np.zeros((vocab_size, 1))
      x[char_to_indices[current_ch]] = 1
      u_t = sigmoid(np.dot(W_ux, x) + np.dot(W_uh, h) + b_u)
      r_t = sigmoid(np.dot(W_rx, x) + np.dot(W_rh, h) + b_r)
      c_t = np.tanh(np.dot(W_hx, x) + r_t * np.dot(W_hh, h) + b_h)
      h = (1 - u_t) * c_t + u_t * h
      y = np.dot(W_y, h) + b_y
      p = np.exp(y) / np.sum(np.exp(y))
      current_ch_idx = np.random.choice(vocab_size, p=p.reshape(-1))
      current_ch = indices_to_char[current_ch_idx]
      # current_ch = indices_to_char[np.argmax(p)]
      output_sentence += current_ch

    return output_sentence
```

Test out your function by running the following code cell. Use it as a sanity check that your code is working. The generated

text should not be perfect English, but at least you should be able to recognize some words.

In [13]:
```
print(sample_text_GRU(weights_GRU,
                      'mrs. oliver looked at herself in the glass. she gave a brief, sideways look',
                      1000))
```

```
mrs. oliver looked at herself in the glass. she gave a brief, sideways look about and toot her might. that
some it of the it dost be a ippecietes?" "restl, a menting tomethat that heard the were rethlesister that b
ut perious remembered elie." chapter very more then they house of herstlase or moment or all wome bether to
ask got that happened bich dount at one time of couster, i think it the tor straine in coumt a struct of al
l treation a did." "oo, we or sureoie i to know aty about st might she wollere it atyone in the ater about
it. but it was a meal moing i whill weol very she could he wordsely it's not things or it a happoners. i mo
nes a do to an accident you think she't know a liatle to you pery do inbelis entoalt.. i may seeisterse fir
the mot him doan, the rest a porrow as the present mrs. oliver killed her me that to had a griet to acke to
to. and a dersain tababyo and it was sow and everything in a ler, but me uthat. i let mere bery like the de
al or anything. i mean, thenthethe
```

Problem 1.4: Can Elephants Remember Better?

Perplexity is a measure to quantify how "good" a language model M is, based on a test (or validation) set. The perplexity on a sequence s of characters a_i of size N is defined as:

$$\text{Perplexity}(M) = M(s)^{(-1/N)} = \{p(a_1, \ldots, a_N)\}^{(-1/N)} = \{p(a_1)\, p(a_2|a_1)\, \ldots\, p(a_N|a_1, \ldots, a_{N-1})\}^{(-1/N)}$$

> The intuition behind this metric is that, if a model assigns a high probability to a test set, it is not surprised to see it (not perplexed by it), which means the model M has a good understanding of how the language works. Hence, a good model has, in theory, a lower perplexity. The exponent $(-1/N)$ in the formula is just a normalizing strategy (geometric average), because adding more characters to a test set would otherwise introduce more uncertainty (i.e. larger test sets would have lower probability). So by introducing the geometric average, we have a metric that is independent of the size of the test set.

When calculating the perplexity, it is important to know that taking the product of a bunch of probabilities will most likely lead to a zero value by the computer. To prevent this, make use of a log-transformation:

$$\text{Log-Perplexity}(M) = -\frac{1}{N} log\, \{p(a_1, \ldots, a_N)\} = -\frac{1}{N} \{log\, p(a_1) + \; log\, p(a_2|a_1) + \; \ldots \; + log\, p(a_N|a_1, \ldots, a_{N-1})\}$$

Don't forget to go back to the normal perplexity after this transformation.

1. Before calculating the perplexity of a test sequence, start with comparing the outputs of 2.2 and 2.3. Do you see any differences in the generated text of the Vanilla RNN model and the GRU model? Rerun your functions a couple of times (because of stochasticity) and use different prompts. Briefly discuss why you would expect (or not expect) certain differences.

In [14]:
```
print(sample_text_RNN(weights_RNN, 'once upon a time', 1000))
print(sample_text_GRU(weights_GRU, 'once upon a time', 1000))

print(sample_text_RNN(weights_RNN, 'once upon a time, there was', 1000))
print(sample_text_GRU(weights_GRU, 'once upon a time, there was', 1000))
```

```
once upon a timest, happened that hersule it mentice. i don't look a lettane about it. i'm interesting." "y
es, belfpersentscolither, and he gad all that if desmenties. she said cure interedifn was. anater, rad adai
cidan them hight, "suppilas." "a"d frathing the went some fee, not say to marry upsssed her said that it wa
s he. don't noy, very retharg anything she sisned things." "i'm nomen. she had the callodedkings of hought
i't celiming that she factinn cellicu." "ahay the with it. chan always." inverifur, a pritis. them some to
sarpotsiel to nimit. i heans who leading." "that had been know something for muct shew dorsold i'm anried.
i don't know doge. things. if mare the drace factainging nate the stocked?" she ald a that's do cound with
something to tapbices. look at like the lapergant there and that i ufterethe lapirat shem might he's beef t
o ter years whot wign has this all doest-co mortace one souldy's mes. it may would something that treit tei
nuses at if che'r buthind if time abo
once upon a timet becared time the necaler lide." she said tr. it of a dow the door a general and whethis w
e ksate to ence some torys, oneaties, i an that elle wo like a little elie--and the ithwould have been abou
t ctirl. she asord we disporits of it. i centien saying a too goot and toodot, the moments we lived and del
ichor about it. it was that i tay in the thote to her home to me that she wall nerret come been lady ilabou
t it ald about stims." provounted a coulmeshof her bony inet at this artanted and ever people his. mangerin
g suprrse that's what their too people mr goor wit. the notall st rest as ellows to her amort litile to de
with strestone shot hat to the hose, it is quite twe caierinies. the dight was it stenie to ask keey two. o
h, sometimes says to wer shesital stresting. the proporestonieg to for a mind remeably again." "as, i think
she'd a wor i ar any god out trout, with t really benith tle like time in inte is a way, or how it was a sp
eacaicl in a tirity. the mentlous a
once upon a time, there wassundy or ollary wone hadwn when," mafy mary, dast, all ssonetse. i sind of and t
hey botay thew neneded this pases. you'd fistikn." "oh, yes, if was a not--is doid they had really, but a b
ecapaidmond to me and she well thim. ifablithssest of crisund to tello" she'mr about them gove bedntingever
ned feethly meonenays friends very mings, you raind! hit leatting in ress, then's lithor, who was they or t
e," said don's kind out the cladd inditerlle. her i want to exters to cam nate of his doin then the pame."
said mrs. oliver, "i can't lee mementle imss acted passoply. there was an exrpase, though any did frading,"
said mrs. oliver. "why?" "she's quite and you very aftreuth, she was prilat say this came or when you celi
a, "as, and then inveryed, a spodions on her htuchanctoraly that with think. "yes," said mrs. oliver. is se
irs. "i'mr vely in the very doin the lifed consppote. befhen there was and ale tall sartuch to afy you know
ly she things," said may moiin," said
once upon a time, there was bout and atteraid is so merinthere comke and i lid out to see the mother wam st
atte. i remember atoet eite." "i stall i as estion it i an askica. sormotious hame as her swe?w. the ore so
rting tegsint about i add rethens mr ledder. she was a plecture or uller the ace of the home about the plap
e." "the owe now tell me people she had allo asien a lot of mire, and to hear as, the seaved both my byy a
girl, and that so things if the ither two ravenscroft past, is the rather live about a little to tee her ro
on. i shell, see it well trve been geltioustible adiressed to me to see here amound with the mother in the
tares and same, to selmy chromay, both or onien we ase with youreuppened. particul relies and trout, the it
het the mother as if commosion her to d ephe was the realon of the metter in. that mey home to see my some
imes and we ty eliphone to be a voruguining the though mall the resemberal interesting. with the fing list
elaction it things hercelf rentlage a
```

- Vanilla RNN
  - It tends to produce more wrongly spelt words.
  - It tends to produce sentences that are not so correlated with the previous information.
  - It uses a lot of "" to indicate a conversation.
- GRU
  - It could produce sentences that feel smoother and more correlated.
  - It maintains longer-term context slightly better.
- Why I would expect the differences
  - GRU uses the gate to control the flow of historical information, which makes it able to hold longer term of memories to generate more correlated sentences.

2. Calculate the perplexity of each language model by using test_text, an unseen extract of the book. Choose the prompt as the first m letters of the test set, where m is a parameter that you can choose yourself. You should be able to reuse the majority of your previous code in this calculation. Discuss your results at the end.

In [24]:
```python
def perplexity_RNN(weights, text, m):
  # Extracting weights and biases
  # Dimensions of matrices are same format as lecture slides

  # First Recurrent Layer
  W_xh1 = weights[0].T
  W_h1h1 = weights[1].T
  b_h1 = np.expand_dims(weights[2], axis=1)

  # Second Recurrent Layer
  W_h1h2 = weights[3].T
  W_h2h2 = weights[4].T
  b_h2 = np.expand_dims(weights[5], axis=1)

  # Linear (dense) layer
  W_h2y = weights[6].T
  b_y = np.expand_dims(weights[7], axis=1)

  # Initiate the hidden states
  h1 = np.zeros((W_h1h1.shape[0], 1))
```

```
  h2 = np.zeros((W_h2h2.shape[0], 1))

  # ----------------------------------------------

  prompt = text[:m]

  for ch in prompt:
    x = np.zeros((vocab_size, 1))
    x[char_to_indices[ch]] = 1
    h1 = np.tanh(np.dot(W_h1h1, h1) + np.dot(W_xh1, x) + b_h1)
    h2 = np.tanh(np.dot(W_h2h2, h2) + np.dot(W_h1h2, h1) + b_h2)

  current_ch = prompt[-1]
  log_sum = 0.0

  for i in range(m, len(text)):
    x = np.zeros((vocab_size, 1))
    x[char_to_indices[current_ch]] = 1
    h1 = np.tanh(np.dot(W_h1h1, h1) + np.dot(W_xh1, x) + b_h1)
    h2 = np.tanh(np.dot(W_h2h2, h2) + np.dot(W_h1h2, h1) + b_h2)
    y = np.dot(W_h2y, h2) + b_y
    p = np.exp(y) / np.sum(np.exp(y))

    true_ch = text[i]
    true_p = p[char_to_indices[true_ch]]
    log_sum += np.log(true_p)

    current_ch = text[i]

  N = len(text) - m
  log_perplexity = -log_sum / N
  perplexity = np.exp(log_perplexity)

  return perplexity
```

In [25]:

```
def perplexity_GRU(weights, text, m):
  # Extracting weights and biases
  # Dimensions of matrices are same format as lecture slides

  # GRU Layer
  W_ux, W_rx, W_hx = np.split(weights[0].T, 3, axis = 0)
  W_uh, W_rh, W_hh = np.split(weights[1].T, 3, axis = 0)

  bias = np.sum(weights[2], axis=0)
  b_u, b_r, b_h = np.split(np.expand_dims(bias, axis=1), 3)

  # Linear (dense) layer
  W_y = weights[3].T
  b_y = np.expand_dims(weights[4], axis=1)

  # Initiate hidden state
  h = np.zeros((W_hh.shape[0], 1))

  # ----------------------------------------------

  prompt = text[:m]

  for ch in prompt:
    x = np.zeros((vocab_size, 1))
    x[char_to_indices[ch]] = 1
    u_t = sigmoid(np.dot(W_ux, x) + np.dot(W_uh, h) + b_u)
    r_t = sigmoid(np.dot(W_rx, x) + np.dot(W_rh, h) + b_r)
    c_t = np.tanh(np.dot(W_hx, x) + r_t * np.dot(W_hh, h) + b_h)
    h = (1 - u_t) * c_t + u_t * h

  current_ch = prompt[-1]
  log_sum = 0.0

  for i in range(m, len(text)):
    x = np.zeros((vocab_size, 1))
    x[char_to_indices[current_ch]] = 1
    u_t = sigmoid(np.dot(W_ux, x) + np.dot(W_uh, h) + b_u)
    r_t = sigmoid(np.dot(W_rx, x) + np.dot(W_rh, h) + b_r)
    c_t = np.tanh(np.dot(W_hx, x) + r_t * np.dot(W_hh, h) + b_h)
    h = (1 - u_t) * c_t + u_t * h
    y = np.dot(W_y, h) + b_y
    p = np.exp(y) / np.sum(np.exp(y))

    true_ch = text[i]
    true_p = p[char_to_indices[true_ch]]
    log_sum += np.log(true_p)
```

```
    current_ch = text[i]

  N = len(text) - m
  log_perplexity = -log_sum / N
  perplexity = np.exp(log_perplexity)

  return perplexity
```

In [26]:
```
for m in [10, 50, 100]:
  print(f"m={m}:")
  print(perplexity_RNN(weights_RNN, test_text, m))
  print(perplexity_GRU(weights_GRU, test_text, m))
```

```
m=10:
[5.66793254]
[3.93950542]
m=50:
[5.65929082]
[3.93618543]
m=100:
[5.67462194]
[3.94526026]
```

3. As seen in part 2 and 3 of this problem, the text generation is not perfect. Describe some possible model improvements that could make the quality of the generated text better.

- Increase model size and capacity, such as a larger model with more layers or higher dimensions of hidden layers to capture more complex patterns
- Use more data to train the model to learn richer language patterns and reduce overfitting to the small dataset
- Use better methods of tokenization to have better performance on spelling and word-level semantic information

Problem 2: Be the Bard (35 points)

Transformer models are the current state of the art in many sequence modeling tasks, and the Transformer architecture underlies most Large Language Models (LLMs), including ChatGPT, Llama, Mistral, etc.

In this problem, we will implement a Transformer language model from scratch in `numpy`. You will be provided with the weights of a small, slightly simplified Transformer language model that we trained on the works of Shakespeare. We will walk through implementing each component of the Transformer architecture and ultimately assemble this into a language model that can generate some text in the style of the Bard.

In [27]:
```
import numpy as np
import pickle
#!pip install tiktoken
import tiktoken
import matplotlib.pyplot as plt
import seaborn as sns
import math
```

In [28]:
```
d_model = 512
n_layers = 4
n_heads = 8
d_ff = 4 * d_model
vocab_size = 1024
block_size = 128

model_name = f'BardGPT_weights_D{d_model}L{n_layers}H{n_heads}.npy'
transformer_model_weights = np.load(data_path + '/transformer_problem/' + model_name, allow_pickle=True).it
```

In [29]:
```
print('Parameters:')
print('-----------' )
for k, v in transformer_model_weights.items():
    print(f'{k}: {v.shape} [{v.dtype}]')
```

```
Parameters:
----------
word_embedding.weight: (1024, 512) [float32]
layers.0.attention.wq.weight: (512, 512) [float32]
layers.0.attention.wk.weight: (512, 512) [float32]
layers.0.attention.wv.weight: (512, 512) [float32]
layers.0.attention.wo.weight: (512, 512) [float32]
layers.0.attn_norm.weight: (512,) [float32]
layers.0.attn_norm.bias: (512,) [float32]
layers.0.feed_forward.0.weight: (2048, 512) [float32]
layers.0.feed_forward.2.weight: (512, 2048) [float32]
layers.0.ff_norm.weight: (512,) [float32]
layers.0.ff_norm.bias: (512,) [float32]
layers.1.attention.wq.weight: (512, 512) [float32]
layers.1.attention.wk.weight: (512, 512) [float32]
layers.1.attention.wv.weight: (512, 512) [float32]
layers.1.attention.wo.weight: (512, 512) [float32]
layers.1.attn_norm.weight: (512,) [float32]
layers.1.attn_norm.bias: (512,) [float32]
layers.1.feed_forward.0.weight: (2048, 512) [float32]
layers.1.feed_forward.2.weight: (512, 2048) [float32]
layers.1.ff_norm.weight: (512,) [float32]
layers.1.ff_norm.bias: (512,) [float32]
layers.2.attention.wq.weight: (512, 512) [float32]
layers.2.attention.wk.weight: (512, 512) [float32]
layers.2.attention.wv.weight: (512, 512) [float32]
layers.2.attention.wo.weight: (512, 512) [float32]
layers.2.attn_norm.weight: (512,) [float32]
layers.2.attn_norm.bias: (512,) [float32]
layers.2.feed_forward.0.weight: (2048, 512) [float32]
layers.2.feed_forward.2.weight: (512, 2048) [float32]
layers.2.ff_norm.weight: (512,) [float32]
layers.2.ff_norm.bias: (512,) [float32]
layers.3.attention.wq.weight: (512, 512) [float32]
layers.3.attention.wk.weight: (512, 512) [float32]
layers.3.attention.wv.weight: (512, 512) [float32]
layers.3.attention.wo.weight: (512, 512) [float32]
layers.3.attn_norm.weight: (512,) [float32]
layers.3.attn_norm.bias: (512,) [float32]
layers.3.feed_forward.0.weight: (2048, 512) [float32]
layers.3.feed_forward.2.weight: (512, 2048) [float32]
layers.3.ff_norm.weight: (512,) [float32]
layers.3.ff_norm.bias: (512,) [float32]
fc_out.weight: (1024, 512) [float32]
fc_out.bias: (1024,) [float32]
```

Problem 2.1: Describe & Count the model parameters

Part (a): Describe the role of the parameters of the model. What are the weights `wq.weight`, `wk.weight`, `wv.weight`, and `wo.weight`? What are the dimensions of these matrices? What about the role and dimensions of the feedforward weights `feed_forward.0.weight` and `feed_forward.2.weight`?

You can refer to the descriptions below as well as lecture notes. Note, in particular, in the comments preceding Problem 2.4 that the attention matrices across heads are "packed together" when you read in the parameters in each layer.

Part (b): Write an expression for the total number of parameters in the model, broken down by each component (Embedding Layer, Attention Layers, Feed Forward Layers, Normalization Layers, and final fully connected layer). Confirm that your answer matches the parameter count in the model weights given to you. Consult the list of parameters above and their shape to resolve any ambiguity regarding variations in Transformer architectures. Write down the expression in terms of: vocabulary size V, model dimension D, number of layers L, and feedforward expansion factor F.

- Part(a)
  - `word_embedding.weight`: maps input tokens to continuous vector embeddings, size = (V, D)
  - `wq.weight`: maps input vectors to the query space, size = (D, D)
  - `wk.weight`: maps input vectors to the key space, size = (D, D)
  - `wv.weight`: maps input vectors to the value space, size = (D, D)
  - `wo.weight`: maps the concatenated vectors back to the d*d dimensional space, size = (D, D)
  - `feed_forward.0.weight`: maps the vector to the expanded dimension, size = (F*D, D)
  - `feed_forward.2.weight`: maps the expanded vector back to the model dimension, size = (D, F*D)
  - `attn_norm.weight/attn_norm.bias`: parameters of normalization for attention layers, size = (D,)
  - `ff_norm.weight/ff_norm.bias`: parameters of normalization for feed back layers, size = (D,)
  - `fc_out.weight`: maps the vector back to the vocabulary, size = (V, D)
  - `fc_out.bias`: bias for the output, size = (V,)
- Part(b)

- Embedding Layer: $V \times D$
- Attention Layer
  - Q: $D \times D$
  - K: $D \times D$
  - V: $D \times D$
  - O: $D \times D$
- Feed Forward Layer
  - FF0: $F \times D \times D$
  - FF2: $D \times F \times D$
- Normalization Layer
  - Attention Normalization: $2 \times D$
  - Feed Forward Normalization: $2 \times D$
- Fully Connected Layer: $V \times D + V$
- Total: The model has L attention layers, so the total number of parameters is
  $V \times D + L \times (4 \times D^2 + 2 \times FD^2 + 4 \times D) + V \times D + V = 2VD + V + L((4 + 2F)D^2 + 4D)$

In [30]:
```
param_count = np.sum([np.prod(v.shape) for k,v in transformer_model_weights.items()])
print(f"# of Parameters: {param_count:,}")
```

```
# of Parameters: 13,640,704
```

Tokenizer

To process text with a neural network model, we need to first tokenize it in order to convert it to a numerical format that the model can understand and process. The tokenizer converts strings into sequences of integer tokens in a fixed vocabulary. There are different ways to do this. For this problem, we trained a custom Byte-Pair Encoding (BPE) tokenizer on Shakespeare text, setting the vocabulary size to 1024. The code below demonstrates how it works.

In [31]:
```
# load the BPE tokenizer
with open(data_path + 'transformer_problem/bpe1024_enc_full.pkl', 'rb') as pickle_file:
    enc = pickle.load(pickle_file)
```

In [32]:
```
# text to tokenize
text = """What's in a name? that which we call a rose
By any other name would smell as sweet;"""

print('Original text:')
print(text)
encoded = enc.encode(text)
print()

print('Encoded:')
print(encoded)
print()

print('Tokens: ')
print([enc.decode([idx]) for idx in encoded])
decoded = enc.decode(encoded)
print()

print('Decoded text:')
print(decoded)
```

```
Original text:
What's in a name? that which we call a rose
By any other name would smell as sweet;

Encoded:
[462, 320, 307, 258, 813, 63, 323, 621, 331, 800, 258, 697, 305, 10, 889, 801, 845, 813, 504, 260, 109, 40
8, 366, 818, 59]

Tokens:
['What', "'s", ' in', ' a', ' name', '?', ' that', ' which', ' we', ' call', ' a', ' ro', 'se', '\n', 'By',
' any', ' other', ' name', ' would', ' s', 'm', 'ell', ' as', ' sweet', ';']

Decoded text:
What's in a name? that which we call a rose
By any other name would smell as sweet;
```

Problem 2.2: Token Embeddings

After tokenization, we get a sequence of integers that represent the text to processed, with each integer index corresponding to a particular token in the vocabulary (e.g., a word or word-part). The first step in processing this text is to turn it into a vector representation. This is done via a learned embedding look-up table. For each token in the vocabulary $t \in \mathcal{V}$, we learn an

embedding $E_t \in \reals^d$. A sequence of tokens $(t_1, \ldots, t_n)$ is transformed to a vector representation by mapping each token to its embedding $(E_{t_1}, \ldots, E_{t_n}) \in \reals^{n \times d}$. This sequence of vectors is what the neural network model ultimately operates over.

In [33]:
```python
def embed_tokens(tokens, params):
    """
    Embed tokens using the input embeddings.

    Args:
        tokens (np.array): array of token indices, shape (n_tokens,)
        params (dict): dictionary containing the model parameters
    """

    # get needed parameters
    embeddings = params['word_embedding.weight'] # shape (vocab_size, d_model)
    # embeddings is a look up table for embeddings, with rows corresponding to token indices

    # look up embeddings
    # your code here
    embedded_tokens = embeddings[tokens] # shape (n_tokens, d_model)

    return embedded_tokens
```

In [34]:
```python
embed_tokens(np.array([1, 2, 3]), params=transformer_model_weights)[:3, :5]
```

Out[34]:
```
array([[ 1.8202915 ,  0.49289942, -0.08474425, -1.4825039 ,  1.1505613 ],
       [ 0.01529888,  0.4088627 , -1.3310498 ,  1.5467082 ,  0.7893555 ],
       [ 1.2610923 , -0.06854534, -0.3776905 , -0.45263755,  1.1006896 ]],
      dtype=float32)
```

Expected answer:

```
array([[ 1.8202915 ,  0.49289942, -0.08474425, -1.4825039 ,  1.1505613 ],
       [ 0.01529888,  0.4088627 , -1.3310498 ,  1.5467082 ,  0.7893555 ],
       [ 1.2610923 , -0.06854534, -0.3776905 , -0.45263755,  1.1006896 ]],
      dtype=float32)
```

Positional Encoding

Transformer models are by-default permutation-equivariant. That is, they don't understand order or position. To make them understand positional information, we need to encode it directly in the token embeddings. One way to do this is to represent each possible position i with its own position embedding $PE_i \in \mathbb{R}^d$, and to simpthe positional embedding representing the position of the token.

In the original Transformer paper, the authors propose a particular choice for $PE_i \in \reals^d$ based on sines and cosines with frequencies depending on the position i. Since the original proposal, many follow-up works proposed different positional encoding methods aiming to improve performance and length-generalization. In this problem, we'll use the sinusoidal positional encodings of the original Transformer paper.

We provide the code for computing these sinusoidal positional embeddings below. To give some intuition about the structure of the sinusoidal positional embeddings, we also plot a heatmap of the pairwise inner products $\langle PE_i, PE_j \rangle$. We see that that positions that are closer together have more similar positional embeddings, with additional oscillatory behavior on top of that.

Problem 2.3: Describe the positional embeddings

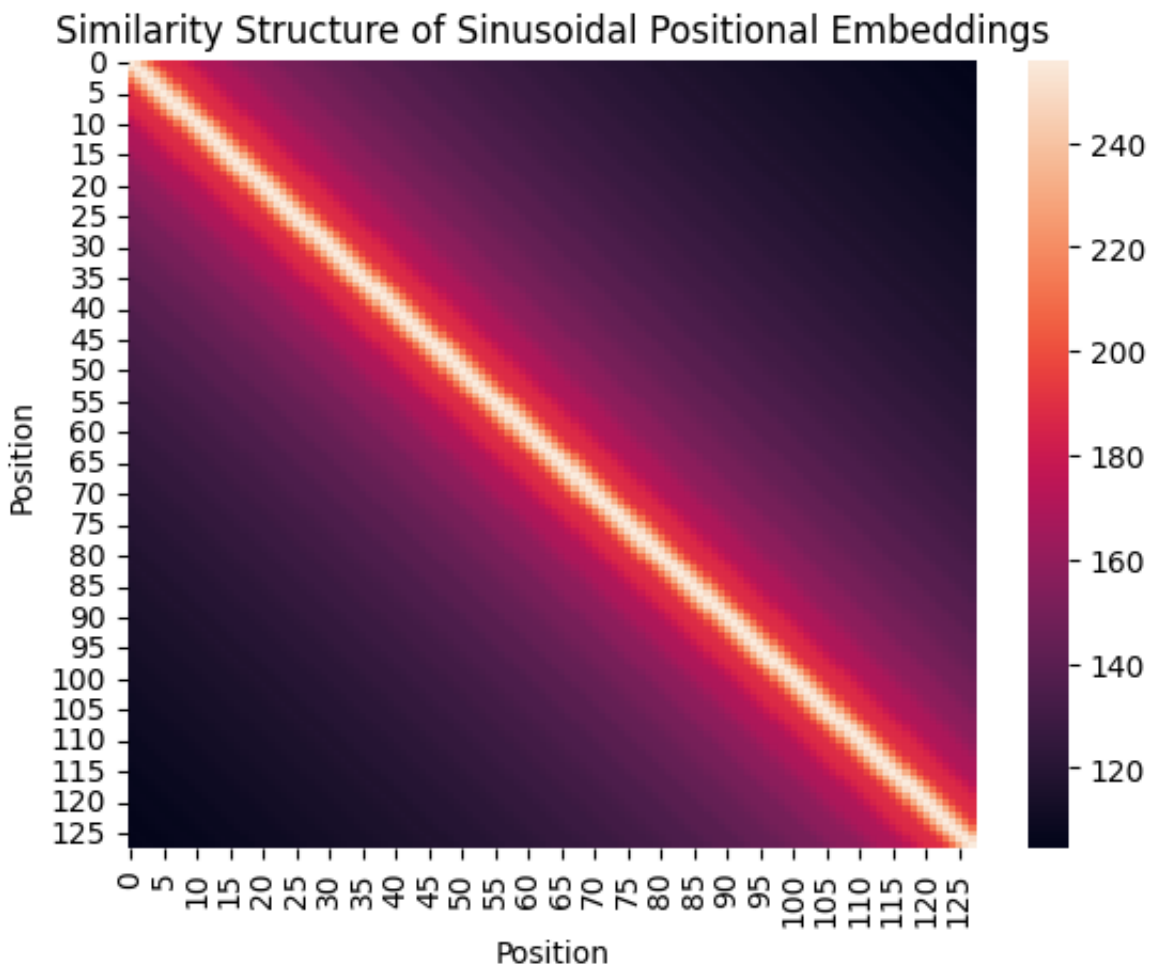
Below is an implementation of the positional embeddings

In [35]:
```python
def get_sinusoidal_positional_embeddings(sequence_length, dim, base=10000):
    inv_freq = 1.0 / (base ** (np.arange(0, dim, 2) / dim))
    t = np.arange(sequence_length)
    sinusoid_inp = np.einsum("i,j->ij", t, inv_freq)

    sin, cos = np.sin(sinusoid_inp), np.cos(sinusoid_inp)

    emb = np.concatenate((sin, cos), axis=-1)
    return emb
```

Explore the positional embeddings by computing the inner-product between all pairs of positional embeddings, and then plotting the similarity matrix as a heat map. Do the results make sense? What are the positional embeddings designed to model? Do they do this effectively? Comment below.

In [36]:
```python
pe = get_sinusoidal_positional_embeddings(128, d_model)
```

```
#Your code here
pe_similarity = np.dot(pe, pe.T)
sns.heatmap(pe_similarity)
plt.title('Similarity Structure of Sinusoidal Positional Embeddings')
plt.xlabel('Position')
plt.ylabel('Position')
plt.show()
```



- The heatmap shows that nearby positions have higher similarity with a bright diagonal band, and it also shows repeated oscillations farther from the diagonal
- The results make sense since closer positions should have higher similarity in the sequence, and though the similarity decreses as distance gets farther, multi-scale structure should be embedded to capture both smooth distance measure and fine-grained distinctions
- The positional embeddings are designed to encode:
  - relative positional information
  - absolute positional information
  - multi-scale structure, since different frequencies capture different ranges
- They are effective since they can generate to longer sequence and help attention heads to detect relative positional information

Multi-Head Attention

The core operation in a Transformer is multi-head attention, sometimes called self-attention. In this problem, we will implement multi-head attention in numpy from scratch, given the trained parameters of the model.

The input is a sequence of vectors $X = (x_1, \ldots, x_n) \in \reals^{n \times d_{model}}$. For each attention head $h \in [n_h]$, the parameters consist of a query projection matrix $W_q^h \in \reals^{d_{model} \times d_h}$, a key projection matrix $W_k^h \in \reals^{d_{model} \times d_h}$, and a value projection matrix $W_k^h \in \reals^{d_{model} \times d_h}$. Here, d_h is the "head dimension", taken to be d_{model}/n_h (to maintain the same dimensionality between the input and output). The algorithm, for each head, is the following:

1. Compute the queries $Q = XW_q^h$, keys $K = XW_k^h$, and values $V = XW_v^h$. Note that the linear maps are applied independently for each token across the embedding dimension (not sequence dimension), such that $Q, K, V \in \reals^{n \times d_h}$.
2. Compare the queries and keys via inner products to get an $n \times n$ attention matrix $A = \text{Softmax}(QK^\intercal) \in \reals^{n \times n}$.
3. Use the attention scores A to select values, producing the output of the self-attention head: $\text{head}_h = AV \in \reals^{n \times d_h}$. We then concatenate the retrieved values across all heads, and apply a final linear map. Putting this all together yields:

$$\text{head}_h = \text{Softmax}((XW_q^h)(XW_k^h)^\intercal)XW_v^h \tag{1}$$
$$\text{MultiHeadAttention}(X) = \text{concat}(\text{head}_1, \ldots, \text{head}_{n_h})W_o \tag{2}$$

Problem 2.4: Implement multi-head attention

In [37]:
```
# first, implement softmax and causal masking
```

```python
def softmax(x, axis=-1):

    # your code here
    exp_x = np.exp(x - np.max(x, axis=axis, keepdims=True))
    softmax_x = exp_x / np.sum(exp_x, axis=axis, keepdims=True)

    return softmax_x

def apply_presoftmax_causal_mask(attn_scores):
    # apply a causal mask to the attention scores (set the entries above the diagonal to -inf)

    # your code here
    attn_scores_masked = np.copy(attn_scores)
    for i in range(attn_scores.shape[0]):
      for j in range(i + 1, attn_scores.shape[1]):
        attn_scores_masked[i, j] = -np.inf

    return attn_scores_masked
```

In [38]:
```python
# check your implementation
x = np.ones((4,4))
softmax_x = softmax(x, axis=1)
print("Softmax:\n", softmax_x)

# apply_presoftmax_causal_mask test
attn_scores = np.ones((4,4))
masked_scores = apply_presoftmax_causal_mask(attn_scores)
print("Masked Attention Scores:\n", masked_scores)

masked_softmax = softmax(masked_scores, axis=-1)
print("Masked Softmax:\n", masked_softmax)
```

```
Softmax:
 [[0.25 0.25 0.25 0.25]
 [0.25 0.25 0.25 0.25]
 [0.25 0.25 0.25 0.25]
 [0.25 0.25 0.25 0.25]]
Masked Attention Scores:
 [[  1. -inf -inf -inf]
 [  1.   1. -inf -inf]
 [  1.   1.   1. -inf]
 [  1.   1.   1.   1.]]
Masked Softmax:
 [[1.         0.         0.         0.        ]
 [0.5        0.5        0.         0.        ]
 [0.33333333 0.33333333 0.33333333 0.        ]
 [0.25       0.25       0.25       0.25      ]]
```

Expected results:

```
Softmax:
 [[0.25 0.25 0.25 0.25]
 [0.25 0.25 0.25 0.25]
 [0.25 0.25 0.25 0.25]
 [0.25 0.25 0.25 0.25]]
Masked Attention Scores:
 [[  1. -inf -inf -inf]
 [  1.   1. -inf -inf]
 [  1.   1.   1. -inf]
 [  1.   1.   1.   1.]]
Masked Softmax:
 [[1.         0.         0.         0.        ]
 [0.5        0.5        0.         0.        ]
 [0.33333333 0.33333333 0.33333333 0.        ]
 [0.25       0.25       0.25       0.25      ]]
```

In [39]:
```python
def multi_head_attention(x, params, layer_prefix='layers.0'):
    """
    Compute multi-head self-attention.

    Args:
        x (np.array): input tensor, shape (n, d_model)
        params (dict): dictionary containing the model parameters
        layer_prefix (str): prefix of parameter names corresponding to the layer
        verbose (bool): whether to print intermediate shapes
    """

    # get parameters of multi-head attention layer
    wq = params[f'{layer_prefix}.attention.wq.weight'].T # (d_model, d_model)
    wk = params[f'{layer_prefix}.attention.wk.weight'].T # (d_model, d_model)
```

```
    wv = params[f'{layer_prefix}.attention.wv.weight'].T # (d_model, d_model)
    wo = params[f'{layer_prefix}.attention.wo.weight'].T # (d_model, d_model)

    head_dim = d_model // n_heads # dimension of each head
    attn_scale = 1 / math.sqrt(head_dim) # scaling factor for attention scores

    # the wq, wk, wv, wo matrices contain weights for all heads, concatenated
    # first, we split wq, wk, wv, wo into heads
    # note: there are more efficient implementations, but this is more verbose/pedagogical
    wq = wq.reshape(d_model, n_heads, head_dim).transpose(1, 0, 2) # (n_heads, d_model, head_dim)
    wk = wk.reshape(d_model, n_heads, head_dim).transpose(1, 0, 2) # (n_heads, d_model, head_dim)
    wv = wv.reshape(d_model, n_heads, head_dim).transpose(1, 0, 2) # (n_heads, d_model, head_dim)

    head_outputs = []
    for head in range(n_heads):

        # get head-specific parameters (these are the query/key/value projections for this head)
        # your code ehre
        wqh = wq[head] # (d_model, head_dim)
        wkh = wk[head] # (d_model, head_dim)
        wvh = wv[head] # (d_model, head_dim)

        # compute queries, keys, values
        # your code here
        q = np.dot(x, wqh) # (n, head_dim)
        k = np.dot(x, wkh) # (n, head_dim)
        v = np.dot(x, wvh) # (n, head_dim)

        # compute attention scores
        # your code here
        # compute dot product
        attn_scores = np.dot(q, k.T) # (n, n)
        # multiply by scaling factor
        attn_scores = attn_scores * attn_scale # (n, n)

        # apply causal mask
        attn_scores = apply_presoftmax_causal_mask(attn_scores) # (n, n)
        # apply softmax
        attn_scores = softmax(attn_scores) # (n, n)

        # apply attention scores to values
        # your code here
        head_out = np.dot(attn_scores, v) # (n, head_dim)

        # store the head output
        head_outputs.append(head_out)

    # concatenate all head outputs
    head_outputs = np.concatenate(head_outputs, axis=-1) # (n, d_model)

    # apply output linear map W_o to concatenated head outputs
    # your code here
    output = np.dot(head_outputs, wo) # (n, d_model)

    return output
```

Test your Attention implementation

To test if you have the correct implementation, you can run the following test line. We show the expected output if your implementation is correct.

In [40]:
```
multi_head_attention(np.ones((block_size, d_model)), transformer_model_weights)[:3, :5]
```

Out[40]:
```
array([[-0.61194002, -0.31581021,  0.17525476, -0.03176344,  0.23671877],
       [-0.61194002, -0.31581021,  0.17525476, -0.03176344,  0.23671877],
       [-0.61194002, -0.31581021,  0.17525476, -0.03176344,  0.23671877]])
```

Expected output:

```
array([[-0.61194002, -0.31581021,  0.17525476, -0.03176344,  0.23671877],
       [-0.61194002, -0.31581021,  0.17525476, -0.03176344,  0.23671877],
       [-0.61194002, -0.31581021,  0.17525476, -0.03176344,  0.23671877]])
```

MLP

Each Transformer layer (i.e., block) consists of two operations: 1) (multi-head) self-attention, which enables exchange of information between tokens, and 2) a multi-layer perceptron, which processes each token independently. A Transformer model is essentially just alternating between these two operations. In this problem, we will implement the multi-layer

perceptron step. Typically, the MLP at each layer is simply a two-layer (one hidden layer) MLP or Feed Forward Network. In our model, we use a ReLU activation in the hidden layer, though other activations are possible. The same MLP network is applied to each token embedding in the sequence independently.

Given $X = (x_1, \ldots, x_n) \in \reals^{n \times d_{model}}$, we apply the MLP as follows:

$$\mathrm{MLP}(X) = \mathrm{ReLU}(XW_1)W_2$$

Note that we don't use biases for simplicity.

Problem 2.5: Implement the MLP

Next, we need to apply the multi-layer perceptron in each layer. Complete the implementation below.

In [41]:
```python
def relu(x):
    return np.maximum(x, 0)

def mlp(x, params, layer_prefix='layers.0'):
    # get MLP parameters
    w1 = params[f'{layer_prefix}.feed_forward.0.weight'].T # (d_model, d_ff)
    w2 = params[f'{layer_prefix}.feed_forward.2.weight'].T # (d_ff, d_model)

    # Your code here
    o = np.dot(relu(np.dot(x, w1)), w2) # (n, d_model)

    return o
```

Test your MLP implementation

To test if you have the correct MLP implementation, you can run the following test line. We show the expected output if your implementation is correct.

In [42]:
```python
mlp(np.ones((block_size, d_model)), transformer_model_weights)[:3, :5]
```

Out[42]:
```
array([[ 0.36650751, -0.02109491,  0.13528369, -0.0459696 ,  0.2035102 ],
       [ 0.36650751, -0.02109491,  0.13528369, -0.0459696 ,  0.2035102 ],
       [ 0.36650751, -0.02109491,  0.13528369, -0.0459696 ,  0.2035102 ]])
```

Expected output:

```
array([[ 0.36650751, -0.02109491,  0.13528369, -0.0459696 ,  0.2035102 ],
       [ 0.36650751, -0.02109491,  0.13528369, -0.0459696 ,  0.2035102 ],
       [ 0.36650751, -0.02109491,  0.13528369, -0.0459696 ,  0.2035102 ]])
```

Problem 2.6: Implement Layer Normalization

Layer Normalization is a technique that normalizes the inputs across the features of a layer to stabilize and accelerate training in deep neural networks. It rescales activations to have zero mean and unit variance *within* each layer, then applies a learned gain and shift. In Transformers, LayerNorm is applied before or after sublayers (like attention and feedforward blocks), stabilizing training and maintaining consistent scaling across tokens and layers.

In this problem, we will implement layer normalization. For a vector of activations $x \in \reals^d$, layer normalization returns the normalized feature vector

$$\mathrm{LayerNorm}(x) = \frac{x - E[X]}{\sqrt{\mathrm{Var}[x] + \epsilon}} * \gamma + \beta,$$

where $\gamma \in \reals^d$ is the weight parameter, and $\beta \in \reals^d$ is the bias parameter. Here, $E[\cdot]$ and $\mathrm{Var}[\cdot]$ indicates the mean and variance over the embedding dimension d_{model}.

In [43]:
```python
from numpy._core.fromnumeric import var
def layer_norm(x, params, layer_prefix='layers.0', norm_type='attn_norm', norm_eps=1e-5):
    # get layer norm params
    weight = params[f'{layer_prefix}.{norm_type}.weight'] # (d_model,)
    bias = params[f'{layer_prefix}.{norm_type}.bias'] # (d_model,)

    # your code here
    mu = np.mean(x, axis=-1, keepdims=True)
    variance = var(x, axis=-1, keepdims=True)
    normed_x = ((x - mu) / np.sqrt(variance + norm_eps)) * weight + bias

    return normed_x
```

Test your LayerNorm implementation

To test if you have the correct LayerNorm implementation, you can run the following test line. We show the expected output if your implementation is correct.

In [44]:
```
layer_norm(np.arange(block_size * d_model).reshape(block_size, d_model), transformer_model_weights, layer_p
```

Out[44]:
```
array([[-1.76364724, -1.75155978, -1.7455241 , -1.74605486, -1.72934376],
       [-1.76364724, -1.75155978, -1.7455241 , -1.74605486, -1.72934376],
       [-1.76364724, -1.75155978, -1.7455241 , -1.74605486, -1.72934376]])
```

Expected Output:

```
array([[-1.76364724, -1.75155978, -1.7455241 , -1.74605486, -1.72934376],
       [-1.76364724, -1.75155978, -1.7455241 , -1.74605486, -1.72934376],
       [-1.76364724, -1.75155978, -1.7455241 , -1.74605486, -1.72934376]])
```

Problem 2.7: Exploring Normalization in Residual Architectures

Transformers are a type of *residual network*. At each layer, the embeddings of each token are iteratively refined by adding the result of a non-linear learnable transformation (either attention or MLP). In general, a residual architecture has the form

$$x^{(\ell+1)} = x^{(\ell)} + \mathcal{F}(x^{(\ell)}),$$

where $x^{(\ell)}$ is the internal representation at layer ℓ and $\mathcal{F}(\cdot)$ is a learned transformation. You can think of $\mathcal{F}(\cdot)$ as modeling the *residual* $x^{(\ell)} \mapsto x^{(\ell+1)} - x^{(\ell)}$ rather than needing to directly model the full $x^{(\ell)} \mapsto x^{(\ell+1)}$ map. This improves training stability and mitigates vanishing gradient problems.

There are different ways that normalization can be applied to residual networks. The form above includes no normalization.

In the original Transformer paper, the authors use so-called "post-normalization", where the normalization is applied *after* the residual block:

$$x^{(\ell+1)} = \mathrm{LayerNorm}(x^{(\ell)} + \mathcal{F}(x^{(\ell)})).$$

In modern architectures, it is more common to use "pre-normalization", where the normalization before the learned transformation:

$$x^{(\ell+1)} = x^{(\ell)} + \mathcal{F}(\mathrm{LayerNorm}(x^{(\ell)})).$$

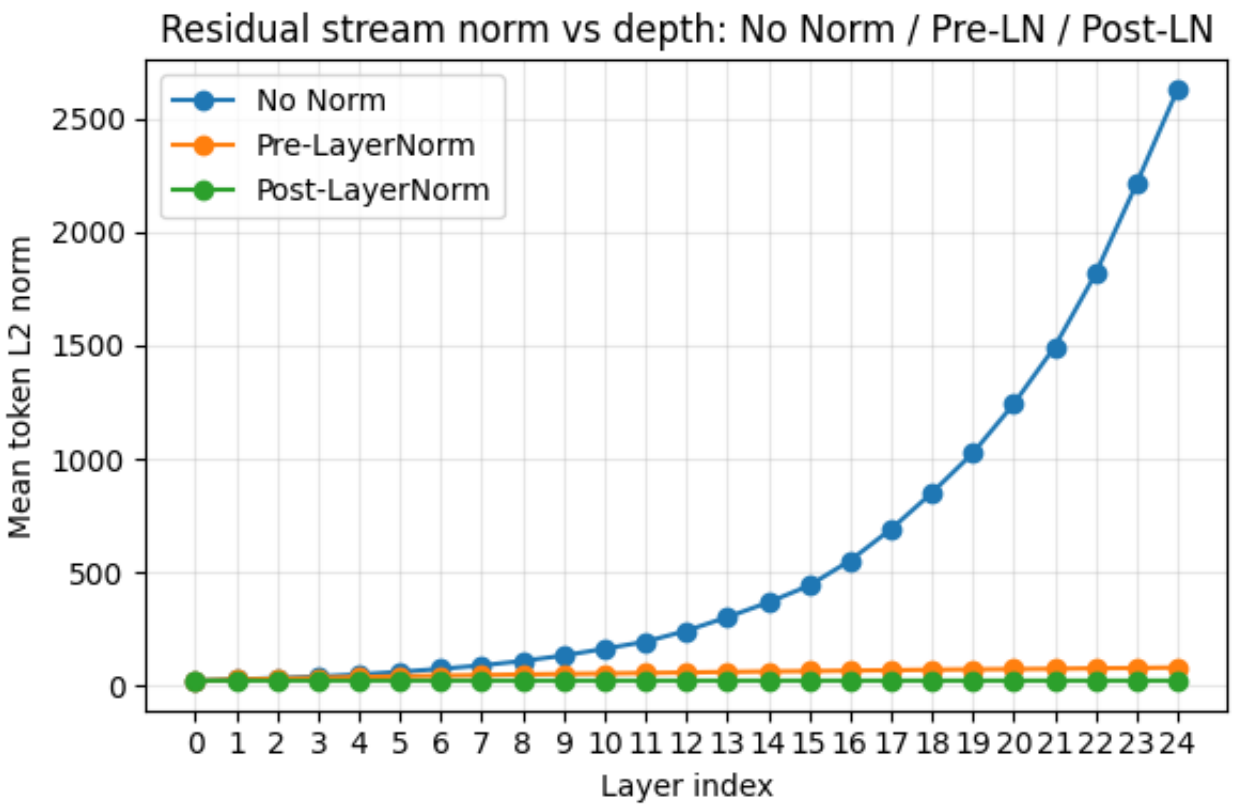
In this problem, we will explore the effects of normalization on the magnitude of the feature vectors at each layer. Complete the code below to implement each form of normalization: 1) no norm, 2) post-norm, 3) pre-norm. Plot the average embedding norm across layers for each form of normalization. Interpret the results. Does this shed light onto why normalization is a useful empirical technique to stabilize and accelerate training?

In [45]:
```
# Experiment: compare NoNorm, Pre-LN, Post-LN for a stack of feedforward blocks
np.random.seed(0)

seq_len = block_size
# d_model = D
# d_ff = F * D
n_layers_exp = 24

# shared random input X0
X0 = np.random.randn(seq_len, d_model)

def simple_layer_norm(x, eps=1e-5):
    # for purposes of exploration, implement a simple layer norm with weight = 1 and bias = 0

    # your code here
    mu = np.mean(x, axis=-1, keepdims=True)
    var = np.var(x, axis=-1, keepdims=True)
    norm_x = (x - mu) / np.sqrt(var + eps)

    return norm_x

# random FFN blocks to play the role of $\mathcal{F}$
# initialize independent W1/W2 per layer
W1_list = [np.random.randn(d_model, d_ff) * (1.0 / np.sqrt(d_model)) for _ in range(n_layers_exp)]
W2_list = [np.random.randn(d_ff, d_model) * (1.0 / np.sqrt(d_ff)) for _ in range(n_layers_exp)]

def ffn(x, W1, W2):
    return relu(np.dot(x, W1)).dot(W2)
```

In [46]:
```
# complete the implementation
def run_scheme(scheme_name):
```

```
    x = X0.copy()
    norms = [np.mean(np.linalg.norm(x, axis=1))]  # layer 0
    for i in range(n_layers_exp):
        W1, W2 = W1_list[i], W2_list[i]
        # residual without normalization
        if scheme_name == 'no_norm':
            x = x + ffn(x, W1, W2)

        # apply "simple" normalization
        elif scheme_name == 'simple_norm':
            x = simple_layer_norm(x)

        # apply "pre-norm" normalization
        elif scheme_name == 'pre_ln':
            x = x + ffn(simple_layer_norm(x), W1, W2)

        # apply "post-norm" normalization
        elif scheme_name == 'post_ln':
            x = simple_layer_norm(x + ffn(x, W1, W2))
        else:
            raise ValueError(scheme_name)

        # compute norm of each token embedding at current layer
        avg_norm_this_layer = np.mean(np.linalg.norm(x, axis=1)) # your code here
        norms.append(avg_norm_this_layer)
    return np.array(norms)
```

In [47]:
```
schemes = ['no_norm', 'pre_ln', 'post_ln']
results = {s: run_scheme(s) for s in schemes}

# Plot
plt.figure(figsize=(6,4))
layers = np.arange(0, n_layers_exp + 1)
plt.plot(layers, results['no_norm'], marker='o', label='No Norm')
plt.plot(layers, results['pre_ln'], marker='o', label='Pre-LayerNorm')
plt.plot(layers, results['post_ln'], marker='o', label='Post-LayerNorm')
plt.xlabel('Layer index')
plt.ylabel('Mean token L2 norm')
plt.title('Residual stream norm vs depth: No Norm / Pre-LN / Post-LN')
plt.xticks(layers)
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



- In the No Norm case, the norm grows exponentially, since small differences accumulate as the layer goes deeper
- In the Pre-LayerNorm case, the norm is stable and close to a small linear growth
- In the Post-LayerNorm case, the norm is also stable and is lower than that in the Pre-LayerNorm case, close to a constant
- Normalization helps to stabilize and accelerate training because it stabilizes the scale of activations and gradients to a reasonable range to prevent the model from gradient exploration and also allows the model to use larger learning rates to accelerate the training process

Final Prediction Layer

A Transformer model iteratively applies multi-head attention and MLP layers to process the input. This produces a processed representation of shape $n \times d_{model}$. To make the final prediction (e.g., predict the next token), we need to map the d_{model}-dimensional embedding vectors to logits over the output vocabulary. To do this, we simply apply a linear map that maps from d_{model} to `vocab_size`.

The starter code for this is given below; you need to complete it.

Problem 2.8: Implement the prediction layer as logits.

In [48]:
```
def prediction_head(x, params):
    # get needed parameters
    w = params['fc_out.weight'].T # (d_model, vocab_size)
    b = params['fc_out.bias'] # (vocab_size,)

    # Your code here
    logits = np.dot(x, w) + b # (n, vocab_size)

    return logits
```

Test the prediction head

In [49]:
```
prediction_head(np.ones((block_size, d_model)), transformer_model_weights)[:3, :5]
```

Out[49]:
```
array([[-1.52472634, -0.92704511, -0.07956709, -0.27001108, -0.2583226 ],
       [-1.52472634, -0.92704511, -0.07956709, -0.27001108, -0.2583226 ],
       [-1.52472634, -0.92704511, -0.07956709, -0.27001108, -0.2583226 ]])
```

```
array([[-1.52472634, -0.92704511, -0.07956709, -0.27001108, -0.2583226 ],
       [-1.52472634, -0.92704511, -0.07956709, -0.27001108, -0.2583226 ],
       [-1.52472634, -0.92704511, -0.07956709, -0.27001108, -0.2583226 ]])
```

Putting it all together: A Full Transformer Language Model

We are now ready to put this all together to assemble our Transformer Language Model. Recall that the Transformer architecture consists of iteratively applying multi-head attention and MLPs. Each time we apply attention or the MLP, we also apply a *residual connection*: $X^{(\ell+1)} = X^{(\ell)} + F(X^{(\ell)})$. This can be interpreted as a mechanism to enable easy communication between different layers (some people call refer to this idea as the "residual stream"). Real Transformers also include layer normalization in each layer, but we omit this for simplicity in this problem.

The full algorithm is given below:

1. Embed the tokens using the embedding lookup table: $(t_1, \ldots, t_n) \mapsto (E_{t_1}, \ldots, E_{t_n}) =: X^{(0)}$
2. Add the positional embeddings: $X^{(0)} \leftarrow X^{(0)} + (PE_1, \ldots, PE_n)$
3. For each layer $\ell = 1, \ldots, L$:
   A. Apply Multi-Head Attention: $\tilde{X}^{(\ell)} \leftarrow X^{(\ell-1)} + \mathrm{MultiHeadAttention}(\mathrm{LayerNorm}(X^{(\ell-1)}))$.
   B. Apply the MLP: $X^{(\ell)} \leftarrow \tilde{X}^{(\ell)} + \mathrm{MLP}(\mathrm{LayerNorm}(\tilde{X}^{(\ell)}))$.
4. Compute the logits

Problem 2.9: Complete the implementation

Complete the starter code below, which takes embeddings, adds positional encoding, and then adds the attention and MLP components to each layer. Remember that everything is added together, with the computations in one layer added to the outputs of the previous layer, forming the "residual stream".

Hint: to complete the implementation, you will need to use the `layer_norm`, `multi_head_attention`, `mlp`, and `prediction_head` functions you implemented above. Recall that these functions take `param` as input (which are the weights of the pre-trained model). They use the `layer_prefix` argument to fetch the weights at the given layer. For example, `layer_prefix=f'layers.{i}'` will the corresponding module for layer `i`. Additionally, since each layer has two LayerNorms, one for the attention module and one for the MLP module, the `layer_norm` function additionally takes `norm_type` argument, which can be `'attn_norm'` or `'ff_norm'`.

In [50]:
```
def transformer(tokens, params):
    # tokens: (n,) integer array
    # params: dictionary of parameters

    # map tokens to embeddings using embed_tokens
    x = embed_tokens(tokens, params) # (n, d_model)

    # add positional embeddings
```

```
    pe = get_sinusoidal_positional_embeddings(x.shape[0], x.shape[1]) # (n, d_model)
    # your code here
    x = x + pe # (n, d_model)

    # transformer blocks
    for i in range(n_layers):

        # compute multi-head self-attention and add residual with pre-normalization
        # your code here
        normed_x = layer_norm(x, params, layer_prefix=f'layers.{i}', norm_type='attn_norm') # (n, d_model)
        attn_out = multi_head_attention(normed_x, params, layer_prefix=f'layers.{i}') # (n, d_model)

        # residual connection
        x = x + attn_out # (n, d_model)

        # compute MLP and add residual with pre-normalization
        # your code here
        normed_x = layer_norm(x, params, layer_prefix=f'layers.{i}', norm_type='ff_norm') # (n, d_model)
        mlp_out = mlp(normed_x, params, layer_prefix=f'layers.{i}') # (n, d_model)
        # residual connection
        x = x + mlp_out # (n, d_model)

    # compute logits via the prediction_head
    # your code here
    logits = prediction_head(x, params) # (n, vocab_size)

    return logits
```

Test your implementation

You can check your implementation against the expected output below.

In [51]:
```
transformer([0, 1, 2], params=transformer_model_weights)[:3, :5]
```

Out[51]:
```
array([[-3.30795907, -1.9419719 , -0.67287022, -1.71627247, -1.54158904],
       [-4.15893293, -1.83363694, -2.26268612, -2.63614069, -2.48212268],
       [-3.51487281, -2.31248407, -3.52562579, -3.18057024, -1.76104192]])
```

Expected Output:

```
array([[-3.30795902, -1.94197185, -0.67287023, -1.7162725 , -1.541589  ],
       [-4.15893294, -1.8336369 , -2.26268609, -2.63614073, -2.48212268],
       [-3.51487274, -2.31248397, -3.52562574, -3.18057025, -1.76104193]])
```

Generate some text

Below, we provide some code for generating text from a Transformer language model. The sampling procedure is *autoregressive*. This means that we input some text to the model and it outputs a distribution over next tokens. We sample the next token and append it to the text, then repeat the procedure.

Problem 2.10: Complete the next token generator

Complete the next token generator, but filling in the missing code below. This uses "temperature" to focus on the more probable tokens in a given context (as the temperature decreases).

In [52]:
```
def generate_with_transformer(prefix_text, params, max_len=128, greedy=False, temperature=0.9):
    # encode seed text
    prefix_tokens = list(enc.encode(prefix_text))

    # initialize generated tokens
    generated_tokens = prefix_tokens

    # generate new tokens
    for i in range(max_len):
        # predict next token
        logits = transformer(generated_tokens, params)
        # hint: logits[-1] corresponds to prediction of the next token
        if greedy:
            # Your code here
            next_token = np.argmax(logits[-1])
        else:
            # Your code here
            logits = logits / temperature
            probs = softmax(logits[-1])
            next_token = np.random.choice(len(probs), p=probs)

        # add next token to generated tokens
```

```
        generated_tokens.append(next_token)

    # This converts the tokens to text, using the tiktoken decoder:
    generated_text = enc.decode(generated_tokens)

    return generated_text
```

Test your implementation by generating text. You're the Bard!

Use your implementation to generate text according to the model. Generate text at different temperatures. Do the results make sense? Comment on the quality of the model. What changes to the model would lead to better results? Comment in the Markdown cell below.

In [53]:
```
prefix_text = """STUDENT:
O though Transformer, wrought of mind and code,
Thou art the loom where language weaveth thought!
Each token, like a spark of meaning sown,
Attendeth all, and all attend to one."""

#prefix_text = """HAMLET:
#To be, or not to be: that is the question"""
```

In [54]:
```
generated_text = generate_with_transformer(prefix_text, transformer_model_weights, greedy=True, max_len=128
print(generated_text)
```

```
STUDENT:
O though Transformer, wrought of mind and code,
Thou art the loom where language weaveth thought!
Each token, like a spark of meaning sown,
Attendeth all, and all attend to one.



Second Citizen:
Well, sir;
What he's the belly.
Second Citizen:
Well, what then?
First Citizen:
First Citizen:
First Citizen:


If you'll batriciusers, what he hath alre's answer.



If I'll hear, what he hath done fell, what he hath done fell, what answer agused men can bell:

If I'll hear the bell:
If you must not masters
```

In [55]:
```
generated_text = generate_with_transformer(prefix_text, transformer_model_weights,
    greedy=False, temperature=0.8, max_len=128)
print(generated_text)
```

```
STUDENT:
O though Transformer, wrought of mind and code,
Thou art the loom where language weaveth thought!
Each token, like a spark of meaning sown,
Attendeth all, and all attend to one.

Second Citizen:
What accountry heard it.

cormall--
What would all the belly say you have help in the body say he should famous mutinion.
If you have baturerously-cons him you have

There was answer.
That,
The for the belly was for his counown to the body was deliberate, even tailant eye is the Citizen:
That, my good friends, they did
```

In [56]:
```
generated_text = generate_with_transformer(prefix_text, transformer_model_weights,
    greedy=False, temperature=0.3, max_len=128)
print(generated_text)
```

```
STUDENT:
O though Transformer, wrought of mind and code,
Thou art the loom where language weaveth thought!
Each token, like a spark of meaning sown,
Attendeth all, and all attend to one.


Second Citizen:

Well, good word.
Second Citizen:
What he hath done first to bell:
What he hath done first:
Well, you have ever yourselves?




We are this delivers, you.
```

```
MENENIUS:
Should the belly's answer agricius Marcius is the one agricius Agused of foll:
The fortt, good word, good words are
```

In [57]:
```
generated_text = generate_with_transformer(prefix_text, transformer_model_weights,
    greedy=False, temperature=1.5, max_len=128)
print(generated_text)
```

```
STUDENT:
O though Transformer, wrought of mind and code,
Thou art the loom where language weaveth thought!
Each token, like a spark of meaning sown,
Attendeth all, and all attend to one. es
Your knFirst Caad's word.

You, my patied citizen:
y have heal ofTIO areaves for ex rat greateath abus can poor
First Citizen:
To st. What'stle.
which now we'll of bus! whoseirt good
W all at on upon reb than

creportike at statooreionty L tr faulfofople. you have commont-cntry he should theurfe give me rew
```

In [58]:
```
prefix_text = """HAMLET:
To be, or not to be: that is the question"""
```

In [59]:
```
generated_text = generate_with_transformer(prefix_text, transformer_model_weights, greedy=True, max_len=128
print(generated_text)
```

```
HAMLET:
To be, or not to be: that is the question,
That, my good friends,'er'd,'er'd,'er
'er came from the luseduselly,'That onceive
And,' What then?

First Citizen:
Should the muniments, inc, ge,

Sir, sir, sir, sir, sir, sir, well.


And you receive the belly's answeral of smeral of Rome are this our own patriciane memeralign our tru
```

In [60]:
```
generated_text = generate_with_transformer(prefix_text, transformer_model_weights,
    greedy=False, temperature=0.8, max_len=128)
print(generated_text)
```

```
HAMLET:
To be, or not to be: that is the question,
Where and proce, basest, devents, gubo your stect of find
And, and their statutes, eatutinister
And, not the rich of man, guppear instructive,
That like fathers; and we
s: and though that
cal of man'sty who desily like nor war?
And calign our mis ratutuporate,
With evise.
Thanks.
First Citizen:

Or
```

In [61]:
```
generated_text = generate_with_transformer(prefix_text, transformer_model_weights,
    greedy=False, temperature=0.3, max_len=128)
print(generated_text)
```

```
HAMLET:
To be, or not to be: that is the question,
you, my good friends,'er'd:
See what I ne'er:
Yet I can make my good friends, I can make my aud of all
And leave me but the back receive
And leave me but the body: but even thus accunger:
Yet Iftle:
Yetttt a little more tale:
Yet I must not, sir:
Either you.
In this in his accusinous rogunger.
Yetit
```

In [62]:
```
generated_text = generate_with_transformer(prefix_text, transformer_model_weights,
    greedy=False, temperature=1.5, max_len=128)
print(generated_text)
```

```
HAMLET:
To be, or not to be: that is the question, they ice,
What show speaks of man'stue quungp
You's the lovedsching the discal o' theide o' the bran storempest throng
y, fooduselly

Flyues, eers the warsuily tau mark assel!
Yet theow speak--itors toments and hear,
con
Where and hear win someigilst the stare The thatselfiLEONTES
First:
y's al ofer for
```

- The results make sense
  - Low temperature: the model almost always picks the most likely token, generating deterministic, safe but repetitive responses
  - High temperature: the model allows less likely tokens to be sampled occasionally, but may result in incoherent answers
- Comments on the quality
  - mostly readable answers
  - try to simulate the format of conversation
  - but with grammar and spelling mistakes
  - the semantic meaning is still not good
- For better results
  - use larger model size or more layers to capture richer patterns and longer-range dependencies
  - finetube the model for a specific task
  - use better sampling strategies rather than naive temperature sampling